String

Wednesday, 18 March 2026 9:35 AM

🔷 1. What is String in Java?

A **String** is an **object** that stores text (sequence of characters).

```
String name = "Rajni";
```

👉 Here, "Rajni" is a string literal stored in memory.

🔷 2. Important Property: Immutable

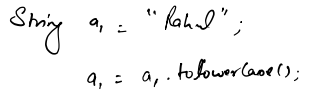
Strings in Java are **immutable**, meaning:

Once created, their value **cannot be changed**.

```
String s = "Hello";
s.concat(" World");
System.out.println(s);
```

👉 Output: Hello

👉 Because original string is not modified.



🔷 3. Ways to Create String

(a) Using String Literal

```
String s1 = "Java";
```

(b) Using new Keyword

```
String s2 = new String("Java");
```

👉 Difference:

- Literal → stored in **String Pool**
- new → stored in **Heap memory**

🔷 4. Common String Methods

Length

```
s.length();
```

Convert Case

```
s.toUpperCase();
s.toLowerCase();
```

Character Access

```
s.charAt(0);
```

Compare Strings

```
s1.equals(s2);
s1.equalsIgnoreCase(s2);
```

Substring

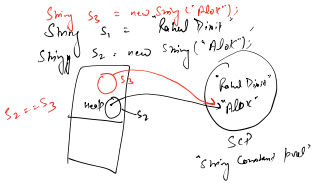
```
s.substring(0, 3);
```

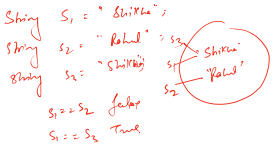
Concatenation

```
s1.concat(s2);
// or
s1 + s2;
```

Contains / Search

```
s.contains("ia");
s.indexOf("a");
```





🔷 5. String Pool Concept

Java uses a **String Pool** to optimize memory.

```
String a = "Java";
String b = "Java";
```

👉 Both a and b point to the **same memory location**.

🔷 6. Difference: == vs .equals()

```
String s1 = "Java";
String s2 = new String("Java");

System.out.println(s1 == s2);     // false
System.out.println(s1.equals(s2)); // true
```

- == → compares memory reference
- .equals() → compares actual content

🔷 7. Mutable Alternatives

Since String is immutable, Java provides:

- StringBuilder (fast, not thread-safe)
- StringBuffer (thread-safe)

🔷 8. Example Program

```
public class Demo {
    public static void main(String[] args) {
        String str = "Java Programming";

        System.out.println("Length: " + str.length());
        System.out.println("Uppercase: " + str.toUpperCase());
        System.out.println("Substring: " + str.substring(0, 4));
    }
}
```

🔷 9. Key Points to Remember

✓ String is an **object**, not primitive

✓ Stored in **String Pool**

✓ **Immutable** in nature

✓ Use .equals() for comparison

✓ Widely used in input/output and data handling

```
public final class java.lang.String implements
java.io.Serializable,
java.lang.Comparable<java.lang.String>,
java.lang.CharSequence, java.lang.constant.Constable,
java.lang.constant.ConstantDesc {
  static final boolean COMPACT_STRINGS;
  public static final java.util.Comparator<java.lang.String>
CASE_INSENSITIVE_ORDER;
  static final byte LATIN1;
  static final byte UTF16;
  public java.lang.String();
  public java.lang.String(java.lang.String);
  public java.lang.String(char[]);
  public java.lang.String(char[], int, int);
  public java.lang.String(int[], int, int);
  public java.lang.String(byte[], int, int, int);
  public java.lang.String(byte[], int);
  public java.lang.String(byte[], int, int,
java.lang.String) throws
java.io.UnsupportedEncodingException;
  public java.lang.String(byte[], int, int,
java.nio.charset.Charset);
  static java.lang.String newStringUTF8NoRepl(byte[], int,
int, boolean);
  static java.lang.String newStringNoRepl(byte[],
java.nio.charset.Charset) throws
java.nio.charset.CharacterCodingException;
  static byte[] getBytesUTF8NoRepl(java.lang.String);
  static byte[] getBytesNoRepl(java.lang.String,
java.nio.charset.Charset) throws
java.nio.charset.CharacterCodingException;
  static int decodeASCII(byte[], int, char[], int, int);
  public java.lang.String(byte[], java.lang.String) throws
java.io.UnsupportedEncodingException;
  public java.lang.String(byte[], java.nio.charset.Charset);
  public java.lang.String(byte[], int, int);
  public java.lang.String(byte[]);
  public java.lang.String(java.lang.StringBuffer);
  public java.lang.String(java.lang.StringBuilder);
  public int length();
  public boolean isEmpty();
  public char charAt(int);
  public int codePointAt(int);
  public int codePointBefore(int);
  public int codePointCount(int, int);
  public int offsetByCodePoints(int, int);
  public void getChars(int, int, char[], int);
  public void getBytes(int, int, byte[], int);
  public byte[] getBytes(java.lang.String) throws
java.io.UnsupportedEncodingException;
  public byte[] getBytes(java.nio.charset.Charset);
  public byte[] getBytes();
  boolean bytesCompatible(java.nio.charset.Charset);
  void copyToSegmentRaw(java.lang.foreign.MemorySegment,
long);
  public boolean equals(java.lang.Object);
  public boolean contentEquals(java.lang.StringBuffer);
  public boolean contentEquals(java.lang.CharSequence);
  public boolean equalsIgnoreCase(java.lang.String);
  public int compareTo(java.lang.String);
  public int compareToIgnoreCase(java.lang.String);
  public boolean regionMatches(int, java.lang.String, int,
int);
  public boolean regionMatches(boolean, int,
java.lang.String, int, int);
  public boolean startsWith(java.lang.String, int);
  public boolean startsWith(java.lang.String);
  public boolean endsWith(java.lang.String);
```

```
  public int hashCode();
  public int indexOf(int);
  public int indexOf(int, int);
  public int indexOf(int, int, int);
  public int lastIndexOf(int);
  public int lastIndexOf(int, int);
  public int indexOf(java.lang.String);
  public int indexOf(java.lang.String, int);
  public int indexOf(java.lang.String, int, int);
  static int indexOf(byte[], byte, int, java.lang.String,
int);
  public int lastIndexOf(java.lang.String);
  public int lastIndexOf(java.lang.String, int);
  static int lastIndexOf(byte[], byte, int,
java.lang.String, int);
  public java.lang.String substring(int);
  public java.lang.String substring(int, int);
  public java.lang.CharSequence subSequence(int, int);
  public java.lang.String concat(java.lang.String);
  public java.lang.String replace(char, char);
  public boolean matches(java.lang.String);
  public boolean contains(java.lang.CharSequence);
  public java.lang.String replaceFirst(java.lang.String,
java.lang.String);
  public java.lang.String replaceAll(java.lang.String,
java.lang.String);
  public java.lang.String replace(java.lang.CharSequence,
java.lang.CharSequence);
  public java.lang.String[] split(java.lang.String, int);
  public java.lang.String[]
splitWithDelimiters(java.lang.String, int);
  public java.lang.String[] split(java.lang.String);
  public static java.lang.String
join(java.lang.CharSequence, java.lang.CharSequence...);
  static java.lang.String join(java.lang.String,
java.lang.String, java.lang.String, java.lang.String[],
int);
  public static java.lang.String
join(java.lang.CharSequence, java.lang.Iterable<? extends
java.lang.CharSequence>);
  public java.lang.String toLowerCase(java.util.Locale);
  public java.lang.String toLowerCase();
  public java.lang.String toUpperCase(java.util.Locale);
  public java.lang.String toUpperCase();
  public java.lang.String trim();
  public java.lang.String strip();
  public java.lang.String stripLeading();
  public java.lang.String stripTrailing();
  public boolean isBlank();
  public java.util.stream.Stream<java.lang.String> lines();
  public java.lang.String indent(int);
  public java.lang.String stripIndent();
  public java.lang.String translateEscapes();
  public <R> R transform(java.util.function.Function<? super
java.lang.String, ? extends R>);
  public java.lang.String toString();
  public java.util.stream.IntStream chars();
  public java.util.stream.IntStream codePoints();
  public char[] toCharArray();
  public static java.lang.String format(java.lang.String,
java.lang.Object...);
  public static java.lang.String format(java.util.Locale,
java.lang.String, java.lang.Object...);
  public java.lang.String formatted(java.lang.Object...);
  public static java.lang.String valueOf(java.lang.Object);
  public static java.lang.String valueOf(char[]);
  public static java.lang.String valueOf(char[], int, int);
  public static java.lang.String copyValueOf(char[], int,
int);
  public static java.lang.String copyValueOf(char[]);
  public static java.lang.String valueOf(boolean);
  public static java.lang.String valueOf(char);
  public static java.lang.String valueOf(int);
  public static java.lang.String valueOf(long);
  public static java.lang.String valueOf(float);
  public static java.lang.String valueOf(double);
  public native java.lang.String intern();
  public java.lang.String repeat(int);
  static void repeatCopyRest(byte[], int, int, int);
  void getBytes(byte[], int, byte);
  void getBytes(byte[], int, int, byte, int);
  java.lang.String(java.lang.AbstractStringBuilder,
java.lang.Void);
  java.lang.String(byte[], byte);
  byte coder();
  byte[] value();
  boolean isLatin1();
  static void checkIndex(int, int);
  static void checkOffset(int, int);
  static int checkBoundsOffCount(int, int, int);
  static void checkBoundsBeginEnd(int, int, int);
  static java.lang.String valueOfCodePoint(int);
  public java.util.Optional<java.lang.String>
describeConstable();
  public java.lang.String
resolveConstantDesc(java.lang.invoke.MethodHandles$Lookup);
  public int compareTo(java.lang.Object);
  public java.lang.Object
resolveConstantDesc(java.lang.invoke.MethodHandles$Lookup)
throws java.lang.ReflectiveOperationException;
  static {};
}
rahul@Rahuls-MacBook-Air /Users %
```

```
public class java.lang.Object {
  public java.lang.Object();
  public final native java.lang.Class<?> getClass();
  public native int hashCode();
  public boolean equals(java.lang.Object);
  protected native java.lang.Object clone() throws
java.lang.CloneNotSupportedException;
  public java.lang.String toString();
  public final native void notify();
  public final native void notifyAll();
  public final void wait() throws java.lang.InterruptedException;
  public final void wait(long) throws java.lang.InterruptedException;
  public final void wait(long, int) throws java.lang.InterruptedException;
  protected void finalize() throws java.lang.Throwable;
}
```

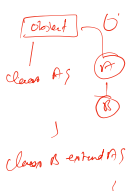
🔷 1. What is Object Class?

- Defined in **java.lang package**
- **Every class in Java directly or indirectly extends Object**

```
class A {
  // implicitly extends Object
}
```

👉 Even if you don't write it, Java automatically does:

```
class A extends Object {
}
```



🔷 2. Why Object Class is Important?

Because it provides **common methods** that every Java object can use.

✔ Code reusability
✔ Standard behavior for all objects
✔ Supports polymorphism

🔷 3. Important Methods of Object Class

1. toString()
Returns string representation of an object.

String s = obj.toString();
- 👉 Default: class name + hashcode
- 👉 Can be overridden

2. equals(Object obj)
Compares two objects.

obj1.equals(obj2);
- 👉 Default: compares memory address
- 👉 Override to compare content

3. hashCode()
Returns hash value of object.

obj.hashCode();
- 👉 Used in HashMap, HashSet

4. getClass()
Returns runtime class of object.

obj.getClass();

5. clone()
Creates a copy of object.

obj.clone();
- 👉 Class must implement Cloneable

6. finalize()
Called before garbage collection.

protected void finalize()
- 👉 Deprecated in modern Java (avoid use)

7. wait(), notify(), notifyAll()
Used in **multithreading (synchronization)**

🔷 4. Example Program

```
class Student {
   int id;
   String name;

   Student(int id, String name) {
      this.id = id;
      this.name = name;
   }

   public String toString() {
      return id + " " + name;
   }
}

public class Test {
   public static void main(String[] args) {
      Student s1 = new Student(1, "Rajni");
      System.out.println(s1);
   }
}
```
- 👉 Output: 1 Rajni

🔷 5. Key Concepts

✓ Inheritance
All classes inherit methods of Object.

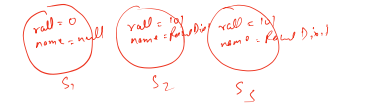
✓ Method Overriding
Commonly overridden:
- toString()
- equals()
- hashCode()

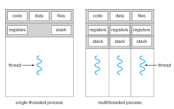
🔷 6. Difference: Object vs Class

Feature	Object Class	User-defined Class
Type	Superclass	Child class
Inheritance	No parent	Extends Object
Purpose	General methods	Specific logic

🔷 7. Key Points to Remember
- ✓ Root class of Java hierarchy
- ✓ Available automatically
- ✓ Provides common methods
- ✓ Used in collections, threading, comparisons

```
Student s1 = new Student();
Student s2 = new Student(101,"Rahul Dixit");
Student s3 = new Student(101,"Rahul Dixit");
```





🧵 Threads in Java
A **Thread** in Java is the smallest unit of execution within a program. It allows a program to perform **multiple tasks simultaneously** (multithreading), improving performance and responsiveness.
Java provides built-in support for multithreading through the **Thread class** and the **Runnable interface**.

🔷 Ways to Create a Thread in Java

1. By Extending Thread Class

```
class MyThread extends Thread {
   public void run() {
      System.out.println("Thread is running...");
   }
}
```

```
package thread;
public class MyThreadDemo {
```

```
r
```

```
public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();
    }
}
```

2. By Implementing Runnable Interface (Recommended ✅)

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread is running...");
    }
}

public class Main {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyRunnable());
        t1.start();
    }
}
```

🔷 Thread Life Cycle

1. **New** → Thread created
2. **Runnable** → Ready to run
3. **Running** → Execution started
4. **Blocked/Waiting** → Waiting for resources
5. **Terminated** → Execution finished

🔷 Advantages of Multithreading ✅

1. Better Performance
- Multiple tasks execute at the same time
- Efficient use of CPU

2. Responsiveness
- UI applications remain responsive (e.g., Android apps)

3. Resource Sharing
- Threads share the same memory space → faster communication

4. Parallelism
- Suitable for tasks like file downloading, gaming, real-time systems

5. Reduced Execution Time
- Tasks are completed faster compared to sequential execution

🔷 Disadvantages of Multithreading ❌

1. Complexity
- Hard to design and debug

2. Synchronization Issues
- Problems like **race condition** may occur

3. Deadlock
- Two or more threads waiting indefinitely

4. Context Switching Overhead
- Switching between threads consumes CPU time

5. Difficult Testing
- Bugs are unpredictable and hard to reproduce

🔷 Example of Multithreading

```
class Demo extends Thread {
    public void run() {
        for(int i = 1; i <= 3; i++) {
            System.out.println("Thread " + i);
        }
    }
}

public class Test {
    public static void main(String[] args) {
        Demo t1 = new Demo();
        Demo t2 = new Demo();

        t1.start();
        t2.start();
    }
}
```

🔷 Key Points to Remember ⭐

- Always use **start()**, not run() directly
- Prefer **Runnable** over extending Thread
- Use **synchronization** when sharing resources
- Threads improve performance but increase complexity

```
  public static void main(String[] args) {
      MyThread m1 = new MyThread();
      //Schedules this thread to begin execution.
      //The thread will execute independently of the current
thread.
//A thread can be started at most once.
      //In particular, a thread can not be restarted after it has
terminated
      m1.start();
      for (int i=-100;i<0;i++)
        System.out.println("Main "+i);

  }
}
```

In Java, **wrapper classes** provide a way to use primitive data types as objects. They are part of the java.lang package and are essential for working with features that require objects, such as the Collections Framework (e.g., ArrayList, HashMap) and Generics. [1, 2, 3, 4]

The 8 Wrapper Classes

Each of Java's eight primitive types has a corresponding wrapper class: [3, 4]

Primitive Type [5, 6, 7, 8, 9]	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Key Concepts

- **Autoboxing:** The automatic conversion that the Java compiler makes between a primitive type and its corresponding wrapper class object (e.g., int to Integer).
- **Unboxing:** The reverse process, where a wrapper object is automatically converted back into its primitive type.
- **Immutability:** All wrapper classes are **final and immutable**, meaning their value cannot be changed once the object is created.
- **Nullability:** Unlike primitives, wrapper classes can hold null values, which is useful for representing absent data in databases or optional fields. [3, 4, 8, 10, 11, 12]

Primary Uses

1. **Collections Framework:** Classes like `ArrayList<Integer>` cannot store primitive int values; they must use the Integer object.
2. **Utility Methods:** They provide useful static methods for data manipulation, such as `Integer.parseInt("123")` for converting strings to integers.
3. **Synchronization:** In multithreading, objects are required to support synchronization, allowing you to use wrapper objects for this purpose.
4. **Serialization:** Converting primitive values into objects is necessary for serialization, which works only with objects. [2, 3, 4, 5, 13, 14, 15]

Performance Note

While wrapper classes add flexibility, they consume more memory and are slower than primitives because they require object instantiation and heap storage. For high-performance calculations or tight loops, using primitives is generally recommended. [3, 11, 16, 17, 18]

How would you like to proceed?

- Would you like to see a **code example** demonstrating autoboxing and unboxing?
- Do you need details on specific **utility methods** (like parseInt or valueOf)?
- Are you interested in how **Integer caching** (for values -128 to 127) affects performance? [11, 19]